CS509 Laboratory Repository — Assignment 02

Repository Overview

This repository contains the implementation and testing framework for **Assignment 02** of the CS509 Software/Programming Laboratory.

The assignment implements two shortest-path algorithms for weighted directed graphs:

1. **Bellman-Ford** — Single Source Shortest Path (SSSP), supporting negative edge weights and reachable negative-cycle detection.
2. **Floyd-Warshall** — All-Pairs Shortest Path (APSP), supporting negative edge weights and negative-cycle detection.

The implementation also includes:

- Weighted adjacency-list graph input.
- Compressed Sparse Row (CSR) representation.
- Matrix-based graph representation for Floyd-Warshall.
- Automatic test-case discovery.
- Reference-output comparison.
- Algorithm-only execution-time measurement.
- Floyd-Warshall/Bellman-Ford cross-checks for the 10- and 100-vertex test cases.
- A simple common driver that asks which algorithm to execute.

The repository does **not** use a separate test framework. Tests are selected by the common wrapper, executed directly, and reported in the terminal.

Student / Pair Details

Field	Details
Student Name	**Rajan Jha**
Entry Number	**[2026AIM1019]**
Assignment	Assignment 02

Field	Details
Assignment Mode	Double / Buddy
Course	CS509 Laboratory

Language and Environment

Programming Language

- **C++17**

Compiler

- GNU `g++`
- The Makefile uses:

```
-std=c++17
-Wall
-Wextra
-O2
```

Build Tool

- GNU Make

Environment Used for Verification

The repository was compiled and tested using:

```
g++ 14.2.0
GNU Make 4.4.1
```

Any compiler with complete C++17 support should be sufficient because the code uses standard C++17 facilities such as `std::filesystem`.

Directory Structure

```
assignment_02/
│
├── driver/
│   └── main.cpp
│
├── src/
│   ├── graph.h
│   ├── graph_io.h
│   ├── graph_io.cpp
│   ├── csr.h
│   ├── csr.cpp
│   ├── bellman_ford.h
│   ├── bellman_ford.cpp
│   ├── floyd_warshall.h
│   ├── floyd_warshall.cpp
│   ├── output.h
│   ├── output.cpp
│   ├── compare.h
│   ├── compare.cpp
│   ├── timer.h
│   ├── runner.h
│   └── runner.cpp
│
├── tests/
│   ├── bellman_ford/
│   │   ├── test_01.txt
│   │   ├── test_02.txt
│   │   ├── ...
│   │   └── test_13.txt
│   │
│   └── floyd_warshall/
│       ├── test_01.txt
│       ├── test_02.txt
│       ├── ...
│       ├── test_10.txt
│       └── test_100.txt
│
├── outputs/
│   ├── bellman_ford/
│   │   └── reference output files
│   │
│   └── floyd_warshall/
│       └── reference output files
│
```

```
├── generated/
│   └── automatically generated test outputs
│
├── driver/main.cpp
├── Makefile
└── README.md
```

Main Folder Purpose

Folder	Purpose
`driver/`	Contains the main program and user-facing menu.
`src/`	Contains graph structures, algorithms, input/output, timing, comparison, and runner code.
`tests/`	Contains all assignment input test cases.
`outputs/`	Contains the expected/reference output for every test case.
`generated/`	Stores output generated during execution. It is created automatically.

Common Wrapper: Build and Usage

Compilation

From the repository root:

```
make
```

This creates:

```
graph_runner
```

To remove compiled object files and the executable:

```
make clean
```

To perform a clean rebuild:

```
make rebuild
```

Execution

Run:

```
./graph_runner
```

The common wrapper presents:

```
Graph Algorithms Test Runner
1. Bellman-Ford
2. Floyd-Warshall
0. Exit
Enter your choice:
```

Select:

```
1
```

to execute all Bellman-Ford test cases.

Select:

```
2
```

to execute all Floyd-Warshall test cases.

Select:

```
0
```

to exit.

The wrapper automatically discovers the `.txt` files in the corresponding test directory. The user does not have to enter individual filenames.

General Testing and Runtime Conventions

For each test case:

1. The input is read from `tests/<algorithm>/`.
2. The selected algorithm is executed.
3. The result is written to `generated/<algorithm>/`.
4. The generated file is compared line-by-line with the corresponding file in `outputs/<algorithm>/`.
5. The test is marked `PASS` only when the generated output exactly matches the reference output.
6. Execution time is measured around the algorithm call only.

Therefore, file loading, output generation, and reference-file comparison are not included in the reported algorithm execution time.

Timing is reported in milliseconds:

```
Execution Time
0.001 ms
```

The reported time can vary between machines and executions, especially for very small inputs.

Assignment 02 — Bellman-Ford and Floyd-Warshall

Assignment Mode

Objective

The objective is to implement and test shortest-path algorithms for weighted directed graphs:

- Bellman-Ford for single-source shortest paths.
- Floyd-Warshall for all-pairs shortest paths.

The implementation must correctly handle:

- Positive edge weights.
- Zero-weight edges.
- Negative edge weights.
- Unreachable vertices.
- Negative-cycle detection.

The implementation also verifies the Floyd-Warshall result against Bellman-Ford for the required 10- and 100-vertex cases.

---

1. Bellman-Ford

Algorithm / Approach

Bellman-Ford computes the shortest distance from a specified source vertex to every other reachable vertex.

Initially:

```
distance[source] = 0
distance[v] = INF    for v != source
```

The algorithm repeatedly relaxes every directed edge:

```
if distance[u] + weight(u,v) < distance[v]:
    distance[v] = distance[u] + weight(u,v)
```

A simple shortest path contains at most `V - 1` edges, so at most `V - 1` complete relaxation passes are required.

After those passes, one additional pass is performed. If a reachable edge can still be relaxed, a reachable negative-weight cycle exists.

The implementation stops early when an entire pass produces no change.

Negative Cycle Handling

If a reachable negative cycle is detected, the program prints:

```
Negative cycle: true
```

and does not print the distance table, following the output format used by the reference files.

Complexity

```
Time:  O(VE)
Space: O(V + E)
```

Bellman-Ford is implemented using CSR so that outgoing edges can be scanned efficiently.

---

2. Floyd-Warshall

Algorithm / Approach

Floyd-Warshall computes shortest paths between every pair of vertices.

The dynamic-programming recurrence is:

```
dist[i][j] = min(
    dist[i][j],
    dist[i][k] + dist[k][j]
)
```

for every intermediate vertex `k`.

The implementation starts from the input adjacency matrix and progressively allows additional intermediate vertices.

After processing all vertices, a negative cycle exists if:

```
dist[i][i] < 0
```

for any vertex `i`.

Negative Cycle Handling

If a negative cycle is detected, the program prints:

```
Negative cycle: true
```

and does not print the distance matrix.

Complexity

```
Time:  O(V^3)
Space: O(V^2)
```

A matrix representation is used because Floyd-Warshall repeatedly requires direct access to:

```
dist[i][k]
dist[k][j]
dist[i][j]
```

---

3. Input Format

Bellman-Ford Input

Bellman-Ford uses a weighted adjacency-list representation:

```
V E
vertex degree neighbour weight neighbour weight ...
vertex degree neighbour weight ...
...
SOURCE source_vertex
```

Example:

```
5 10
0 2 1 6 3 7
1 3 2 5 3 8 4 -4
2 1 1 -2
3 2 2 -3 4 9
4 2 0 2 2 7
SOURCE 0
```

The graph is directed. Every listed adjacency entry represents one directed edge.

The source vertex must satisfy:

```
0 <= source < V
```

Floyd-Warshall Input

Floyd-Warshall uses a `V x V` matrix:

```
V
row 0
row 1
...
row V-1
```

Example:

```
5
0 3 8 INF -4
INF 0 INF 1 7
INF 4 0 INF INF
2 INF -5 0 INF
INF INF INF 6 0
```

`INF` represents the absence of a direct edge.

The diagonal is required to contain zero.

For the current implementation, the source for a Floyd-Warshall input is set internally to vertex `0`; Floyd-Warshall itself computes all source-destination pairs.

---

4. Graph Representations

Graph

The `Graph` structure stores the weighted adjacency list:

```
vector<vector<int>> adj;
vector<vector<int>> weights;
```

For a vertex `u` :

```
adj[u]
```

contains destination vertices and:

```
weights[u]
```

contains their corresponding edge weights.

CSRGraph

The adjacency list is converted to CSR:

```
row_ptr
col_idx
weights
```

For vertex `u` , its outgoing edges are stored in:

```
for (int i = row_ptr[u];
     i < row_ptr[u + 1];
     ++i)
```

CSR is used for Bellman-Ford because the algorithm repeatedly scans all edges.

MatrixGraph

Floyd-Warshall uses:

```
vector<vector<long long>> matrix;
```

This provides direct `O(1)` matrix access.

---

5. Floyd-Warshall / Bellman-Ford Cross-Check

For Floyd-Warshall test cases with:

```
V = 10
```

or:

```
V = 100
```

the runner performs an additional verification.

The matrix is converted into:

```
MatrixGraph
    ↓
Graph
    ↓
CSRGraph
    ↓
Bellman-Ford
```

Bellman-Ford is executed once from every source vertex.

For every pair `(i,j)` the following must hold:

```
BellmanFord(source=i).distance[j]
==
FloydWarshall.distance[i][j]
```

The current test suite contains:

- One 10-vertex Floyd-Warshall case.
- One 100-vertex Floyd-Warshall case.

Both cross-checks passed during verification.

For negative-cycle cases, the cross-check is not used because the reference output reports the negative cycle instead of a finite distance matrix.

---

6. Source Files and Helper Functions

File	Purpose
`driver/main.cpp`	Displays the common menu and invokes the selected algorithm runner.
`src/graph.h`	Defines `Graph`, `CSRGraph`, `MatrixGraph`, and result structures.
`src/graph_io.h/.cpp`	Reads adjacency-list and matrix input files.
`src/csr.h/.cpp`	Converts `Graph` into CSR representation.
`src/bellman_ford.h/.cpp`	Implements Bellman-Ford and negative-cycle detection.
`src/floyd_warshall.h/.cpp`	Implements Floyd-Warshall and negative-cycle detection.
`src/output.h/.cpp`	Writes algorithm results in the required output format.
`src/compare.h/.cpp`	Performs exact line-by-line comparison between expected and generated output.
`src/timer.h`	Provides high-resolution execution-time measurement.
`src/runner.h/.cpp`	Discovers tests, executes algorithms, creates generated outputs, compares results, and prints summaries.
`Makefile`	Builds and cleans the complete project.

---

7. Compilation

Compile the complete project:

```
make
```

Equivalent compiler configuration from the Makefile:

```
g++ -std=c++17 -Wall -Wextra -O2
```

The final executable is:

```
./graph_runner
```

---

8. Execution

Run:

```
./graph_runner
```

For Bellman-Ford:

```
Enter your choice: 1
```

For Floyd-Warshall:

```
Enter your choice: 2
```

The runner automatically executes every `.txt` test file in the selected algorithm's test directory.

---

9. Output Format

Bellman-Ford Normal Output

Example:

```
Algorithm: Bellman-Ford
Source: 0
Vertex Distance
0 0
1 2
2 4
```

```
3 7
4 -2
Negative cycle: none
```

Bellman-Ford Negative-Cycle Output

```
Algorithm: Bellman-Ford
Source: 0
Negative cycle: true
```

Floyd-Warshall Normal Output

Example:

```
Algorithm: Floyd-Warshall
Distance matrix:
0 1 -3 2 -4
3 0 -4 1 -1
7 4 0 5 3
2 -1 -5 0 -2
8 5 1 6 0
Negative cycle: none
```

Floyd-Warshall Negative-Cycle Output

```
Algorithm: Floyd-Warshall
Negative cycle: true
```

---

10. Test Cases

Bellman-Ford Test Suite

Test File	V	E	Source	Negative Cycle

Test File	V	E	Source	Negative Cycle
test_01.txt	5	10	0	No
test_02.txt	3	3	0	Yes
test_03.txt	5	3	0	No
test_04.txt	1	0	0	No
test_05.txt	5	2	0	No
test_06.txt	6	6	0	No
test_07.txt	5	6	0	No
test_08.txt	6	5	0	No
test_09.txt	6	5	0	Yes
test_10.txt	7	5	0	No
test_11.txt	6	6	0	No
test_12.txt	8	7	2	No
test_13.txt	6	7	0	No

Floyd-Warshall Test Suite

Test File	V	E	Source	Negative Cycle
test_01.txt	5	9	N/A	No
test_02.txt	3	3	N/A	Yes
test_04.txt	4	4	N/A	No
test_05.txt	5	7	N/A	No
test_06.txt	5	7	N/A	No
test_07.txt	4	4	N/A	No
test_08.txt	5	5	N/A	No
test_09.txt	6	6	N/A	Yes
test_10.txt	10	22	N/A	No
test_100.txt	100	220	N/A	No
test_11.txt	8	9	N/A	No
test_12.txt	5	5	N/A	Yes

Test File	V	E	Source	Negative Cycle
`test_13.txt`	6	4	N/A	Yes

For Floyd-Warshall, `E` is the number of finite off-diagonal entries in the input matrix.

---

11. Required README Result Tables

The following results were obtained by compiling the supplied repository with:

```
g++ 14.2.0
GNU Make 4.4.1
```

with optimization:

```
-O2
```

The actual output files generated by the program matched the corresponding reference files exactly for every listed test.

> **Timing note:** Algorithm execution times are machine- and run-dependent. The values below are the measurements from the verification run used to prepare this README. Only the algorithm call is timed.

11.1 Bellman-Ford Results

Algorithm	Test File	Vertices	Edges	Source	Negative Cycle	Expected Output	Actual Output	Time	Status
Bellman-Ford	`test_01.txt`	5	10	0	No	`0, 2, 4, 7, -2`	Exact match	0.001 ms	PASS
Bellman-Ford	`test_02.txt`	3	3	0	Yes	Negative cycle: true	Exact match	0.000 ms	PASS
Bellman-Ford	`test_03.txt`	5	3	0	No	`0, 4, 7, INF, INF`	Exact match	0.000 ms	PASS

Algorithm	Test File	Vertices	Edges	Source	Negative Cycle	Expected Output	Actual Output	Time	Status
Bellman-Ford	test 04.txt	1	0	0	No	0	Exact match	0.000 ms	PASS
Bellman-Ford	test 05.txt	5	2	0	No	0, 2, 5, INF, INF	Exact match	0.000 ms	PASS
Bellman-Ford	test 06.txt	6	6	0	No	0, 3, 1, 4, 7, INF	Exact match	0.000 ms	PASS
Bellman-Ford	test 07.txt	5	6	0	No	0, -2, 2, 1, 2	Exact match	0.000 ms	PASS
Bellman-Ford	test 08.txt	6	5	0	No	0, 3, 1, 3, 10, INF	Exact match	0.000 ms	PASS
Bellman-Ford	test 09.txt	6	5	0	Yes	Negative cycle: true	Exact match	0.000 ms	PASS
Bellman-Ford	test 10.txt	7	5	0	No	0, 2, 5, INF, INF, INF, INF	Exact match	0.000 ms	PASS
Bellman-Ford	test 11.txt	6	6	0	No	0, 0, 0, 0, -1, 1	Exact match	0.000 ms	PASS
Bellman-Ford	test 12.txt	8	7	2	No	INF, INF, 0, 4, 6, 12, 10, INF	Exact match	0.000 ms	PASS
Bellman-Ford	test 13.txt	6	7	0	No	0, -1, -1, 1, 1, 2	Exact match	0.000 ms	PASS

Bellman-Ford summary: 13/13 tests passed.

11.2 Floyd-Warshall Results

For normal Floyd-Warshall cases, the expected output is the complete `V x V` shortest-distance matrix stored in the corresponding file under `outputs/floyd_warshall/`. For large matrices, the README identifies the matrix dimensions rather than reproducing hundreds or thousands of matrix values inline.

Algorithm	Test File	Vertices	Edges	Source	Negative Cycle	Expected Output	Actual Output	Time	Status
Floyd-Warshall	test_01.txt	5	9	N/A	No	5x5 distance matrix	Exact match	0.001 ms	PASS
Floyd-Warshall	test_02.txt	3	3	N/A	Yes	Negative cycle: true	Exact match	0.000 ms	PASS
Floyd-Warshall	test_04.txt	4	4	N/A	No	4x4 distance matrix	Exact match	0.001 ms	PASS
Floyd-Warshall	test_05.txt	5	7	N/A	No	5x5 distance matrix	Exact match	0.001 ms	PASS
Floyd-Warshall	test_06.txt	5	7	N/A	No	5x5 distance matrix	Exact match	0.001 ms	PASS
Floyd-Warshall	test_07.txt	4	4	N/A	No	4x4 distance matrix	Exact match	0.000 ms	PASS
Floyd-Warshall	test_08.txt	5	5	N/A	No	5x5 distance matrix	Exact match	0.000 ms	PASS
Floyd-Warshall	test_09.txt	6	6	N/A	Yes	Negative cycle: true	Exact match	0.001 ms	PASS

Algorithm	Test File	Vertices	Edges	Source	Negative Cycle	Expected Output	Actual Output	Time	Status
Floyd-Warshall	test_10.txt	10	22	N/A	No	10x10 distance matrix	Exact match; BF cross-check PASS	0.002 ms	PASS
Floyd-Warshall	test_100.txt	100	220	N/A	No	100x100 distance matrix	Exact match; BF cross-check PASS	0.453 ms	PASS
Floyd-Warshall	test_11.txt	8	9	N/A	No	8x8 distance matrix	Exact match	0.001 ms	PASS
Floyd-Warshall	test_12.txt	5	5	N/A	Yes	Negative cycle: true	Exact match	0.000 ms	PASS
Floyd-Warshall	test_13.txt	6	4	N/A	Yes	Negative cycle: true	Exact match	0.001 ms	PASS

Floyd-Warshall summary: 13/13 tests passed.

Bellman-Ford cross-check summary:

```
Cross-checks performed: 2
Cross-checks passed:    2
Cross-checks failed:    0
```

The two cross-checks correspond to:

```
test_10.txt   V = 10
test_100.txt  V = 100
```

---

12. Detailed Expected Outputs

The complete expected outputs are stored in:

```
outputs/bellman_ford/
outputs/floyd_warshall/
```

The generated outputs are stored in:

```
generated/bellman_ford/
generated/floyd_warshall/
```

For example:

```
outputs/bellman_ford/test_01.txt
generated/bellman_ford/test_01.txt
```

The two files are expected to be identical after a successful test.

---

13. Verification Method

The comparison function performs an exact line-by-line comparison.

Conceptually:

```
Expected file
      |
      v
read line
      |
      v
compare with generated line
      |
      +---- different ----> FAIL
      |
      v
continue
      |
      v
end of both files
      |
```

```
      v
PASS
```

A difference in:

- number of lines,
- line contents,
- spacing within a line,

causes the test to fail.

This makes the result verification deterministic with respect to the reference output files.

---

14. Runtime Measurement

The `Timer` class uses:

```
std::chrono::high_resolution_clock
```

For Bellman-Ford, the timer surrounds:

```
BellmanFord(csr, source);
```

For Floyd-Warshall, the timer surrounds:

```
FloydWarshall(graph);
```

Therefore the reported time excludes:

- input-file parsing,
- graph/CSR construction,
- output-file writing,
- reference-output comparison,
- Floyd-Warshall cross-check execution.

For Floyd-Warshall 10- and 100-vertex cases, the additional Bellman-Ford cross-check is also outside the recorded Floyd-Warshall execution time.

---

15. Why CSR Is Used for Bellman-Ford

Bellman-Ford repeatedly scans every directed edge.

The CSR representation stores the graph compactly as:

```
row_ptr
col_idx
weights
```

For a vertex `u` :

```
row_ptr[u]
```

and:

```
row_ptr[u + 1]
```

identify the range of outgoing edges in `col_idx` and `weights` .

Thus the implementation can efficiently perform:

```
for (int u = 0; u < V; ++u)
{
    for (int i = row_ptr[u];
         i < row_ptr[u + 1];
         ++i)
    {
        // relax edge
    }
}
```

This gives the expected `O(VE)` Bellman-Ford complexity while using `O(V+E)` graph storage.

---

16. Why a Matrix Is Used for Floyd-Warshall

Floyd-Warshall repeatedly accesses arbitrary pairs:

```
dist[i][j]
dist[i][k]
dist[k][j]
```

A matrix gives direct access:

```
matrix[i][j]
```

in `O(1)` time.

Therefore a matrix is more natural for Floyd-Warshall than CSR.

The Floyd-Warshall matrix is nevertheless converted into the existing graph/CSR pipeline for the required Bellman-Ford cross-check on the 10- and 100-vertex cases.

---

17. References

The following standard references were used for understanding the algorithms and their complexity:

1. Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein, **Introduction to Algorithms**, sections covering Bellman-Ford and Floyd-Warshall.
2. Robert Sedgewick and Kevin Wayne, **Algorithms**, shortest-path algorithms.
3. Course/assignment specification provided for CS509 Laboratory.
4. Standard C++17 library documentation for `std::vector`, `std::filesystem`, and `std::chrono`.

No external library is required for the implementation.

---

18. Final Verification Summary

The complete repository was compiled successfully using:

```
g++ 14.2.0
GNU Make 4.4.1
C++17
-O2
```

Final test results:

Algorithm	Tests	Passed	Failed
Bellman-Ford	13	13	0
Floyd-Warshall	13	13	0
Total	**26**	**26**	**0**

Additional verification:

```
Floyd-Warshall V=10  -> Bellman-Ford cross-check: PASS
Floyd-Warshall V=100 -> Bellman-Ford cross-check: PASS
```

Therefore, all **26 supplied algorithm test cases passed**, and both required large-graph cross-checks passed.